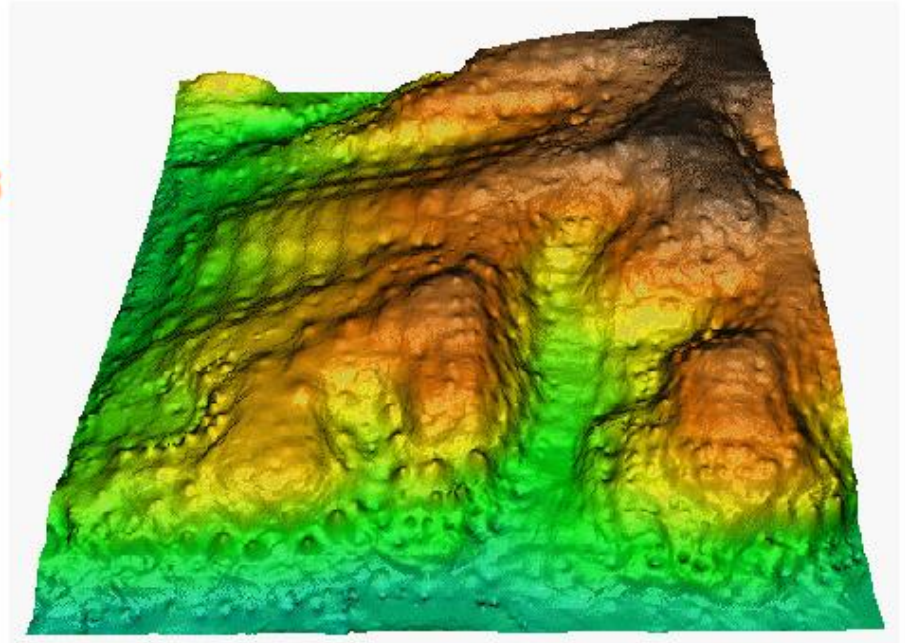
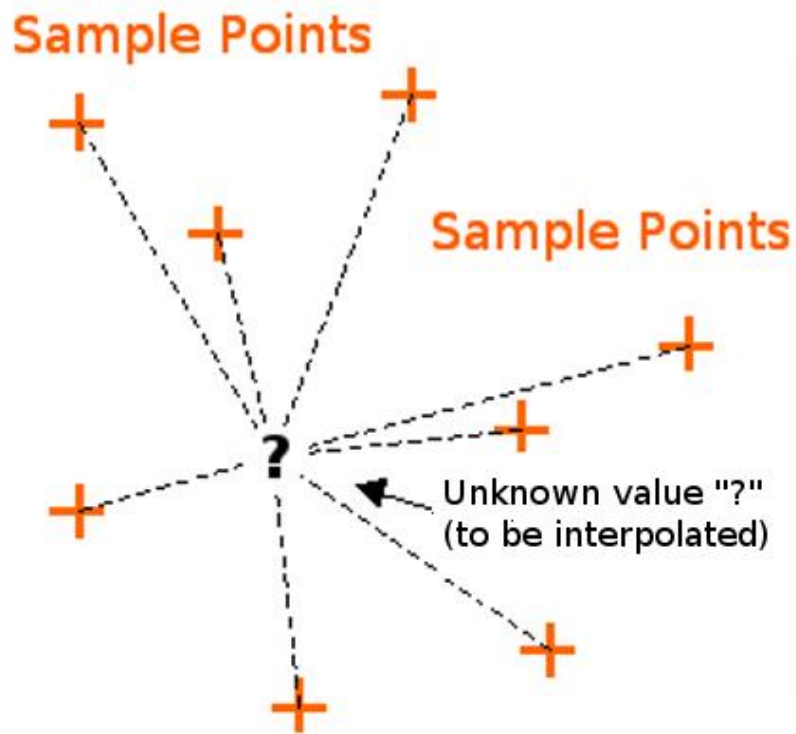


Lecture 17 – Spatial Statistics II



Today's learning outcome

- Apply IDW interpolation and explain the role of the power parameter p
- Describe natural neighbor interpolation and its geometry
- Compute and interpret an empirical variogram
- Identify nugget, sill, and range from a variogram plot
- Fit a parametric variogram model (Spherical, Gaussian, ...)
- Run Ordinary and Universal Kriging using pykrige

Why Spatial Interpolation?

We often have incomplete spatial sampling

→ measurements exist only at discrete, non-random points

Goal: estimate values across the whole landscape

→ fill in the gaps between known points

Practical reasons to interpolate

→ Avoid NA issues in downstream models or functions

→ Enable spatial predictions for management decisions

The Meuse Dataset

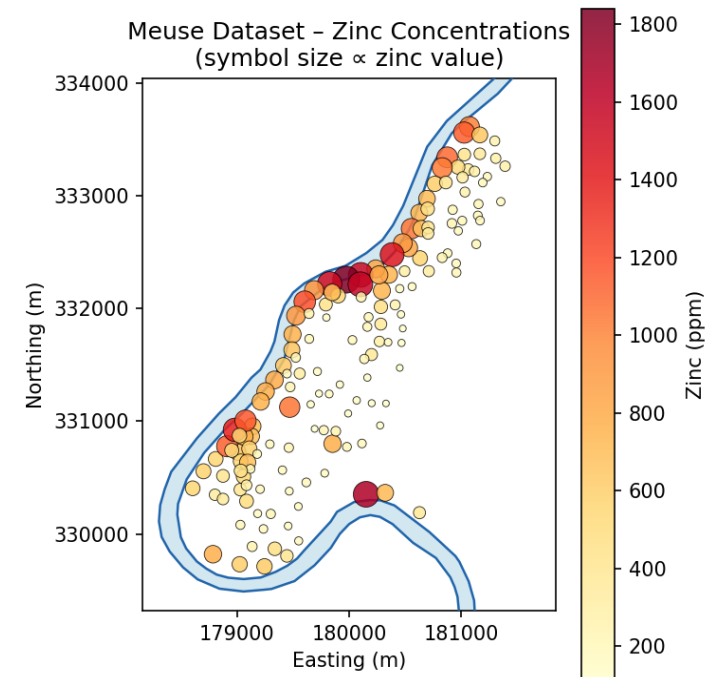
155 sample points along the Meuse river floodplain (Netherlands)

- Variables: zinc, cadmium, copper, lead concentrations (ppm)
elevation, distance from river, soil type, land use

Zinc ranges from 113 to 1839 ppm —
strongly influenced by proximity to
river

We will:

- Map zinc values (symbol size $\propto$ value)
- Interpolate zinc across the full study region
- Model the spatial correlation structure



Load Data in Python

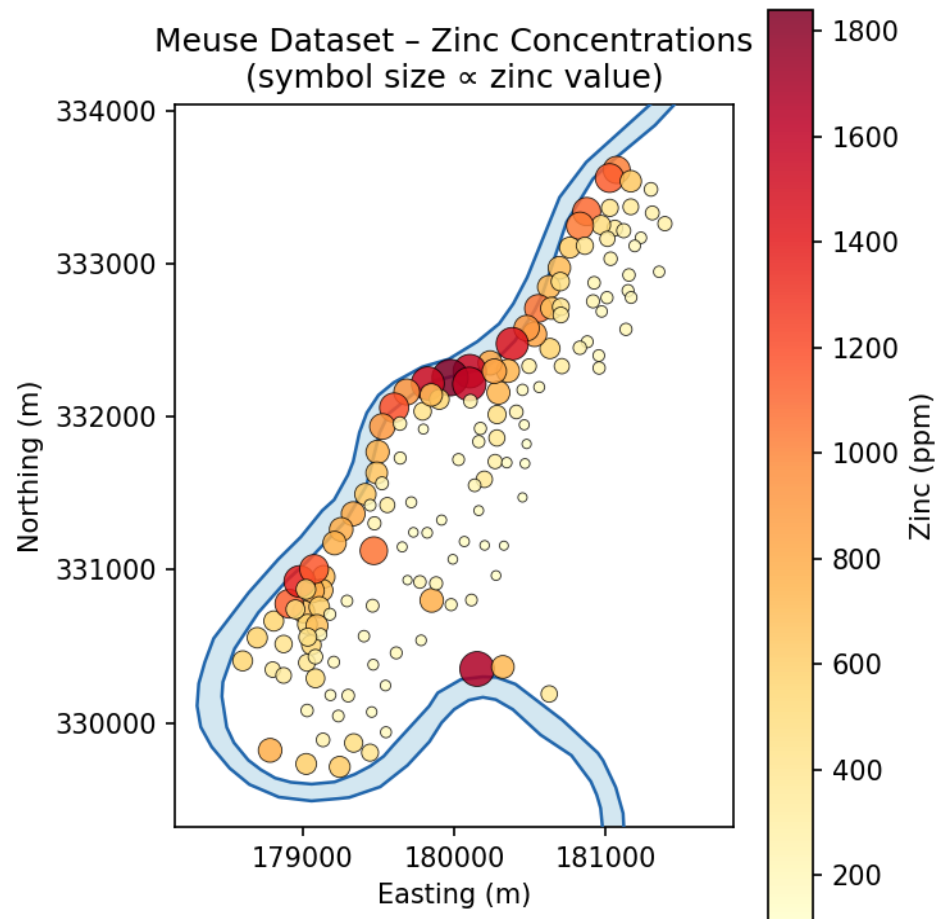
```
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point

# Load the meuse dataset
df = pd.read_csv("meuse.csv")
print(df.shape)          # (155, 14)
print(df.columns.tolist())

# Convert to GeoDataFrame (Dutch RD projection, EPSG:28992)
gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df.x, df.y),
    crs="EPSG:28992"
)

# Quick look at zinc
print(df["zinc"].describe())
```

Zinc Concentrations Along the Meuse River



Symbol size and color $\propto$ zinc value

High zinc near river $\rightarrow$ flood-deposit contamination

Gradient decreases with distance from river

Ranges from 113 to 1839 ppm

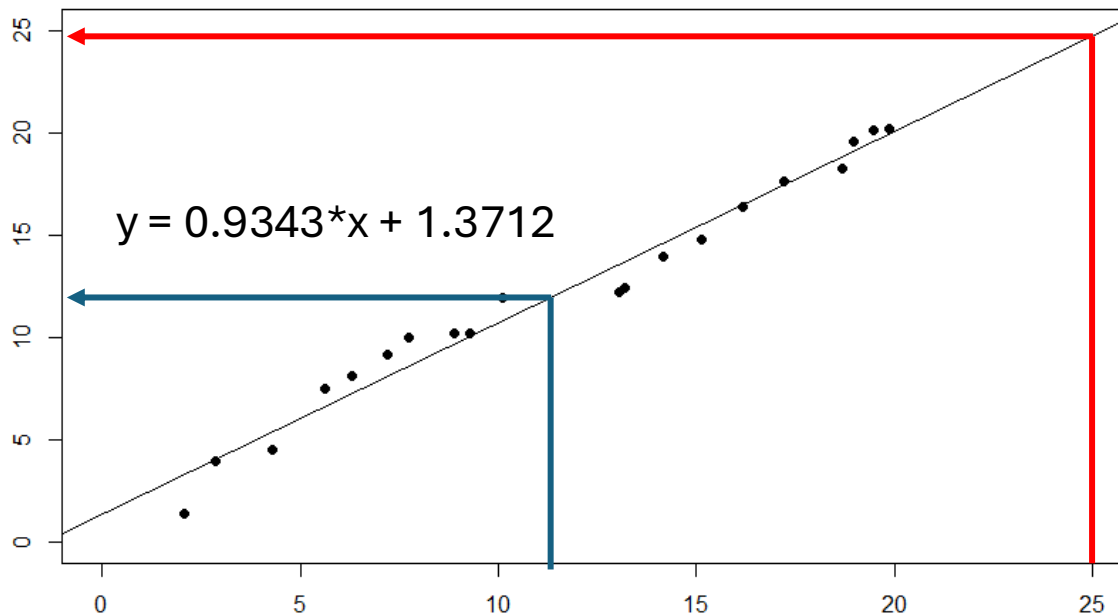
Interpolation

Interpolation: projection of unknown values within the range of observed values

Extrapolation: projection of unknown values outside the range of observed values

Key distinction:

- Interpolation stays between known points — generally safer
- Extrapolation extends beyond — higher uncertainty



Example: a fitted regression line is used for interpolation (blue) or extrapolation (red) depending on where you predict

Interpolation Applications

Geology / Mining

- Estimation of ore grades over mineable units from drill-hole data

Environmental Science

- Air quality, soil pollution, groundwater head, hydraulic conductivity
- Vegetation distribution from monitoring networks

General

- Estimation of spatial averages from spatially correlated data

Interpolation Terms

Random field $Z(s)$: a spatially varying quantity, measured at discrete points

s — spatial location (x, y coordinates)

s_0 — target location where we want to predict Z

B_0 — region of target locations (the prediction grid)

Two problems to solve:

1. Estimate a reliable $Z(s_0)$ — the predicted value
2. Quantify uncertainty in that estimate

Interpolation Techniques

We will cover four main methods:

1. Inverse Distance Weighting (IDW)

- assumes spatial autocorrelation; weights by $\frac{1}{distance^p}$

2. Natural Neighbor Interpolation

- weights by Thiessen polygon area overlap

3. Variogram Modelling

- parametric model of spatial correlation

4. Kriging

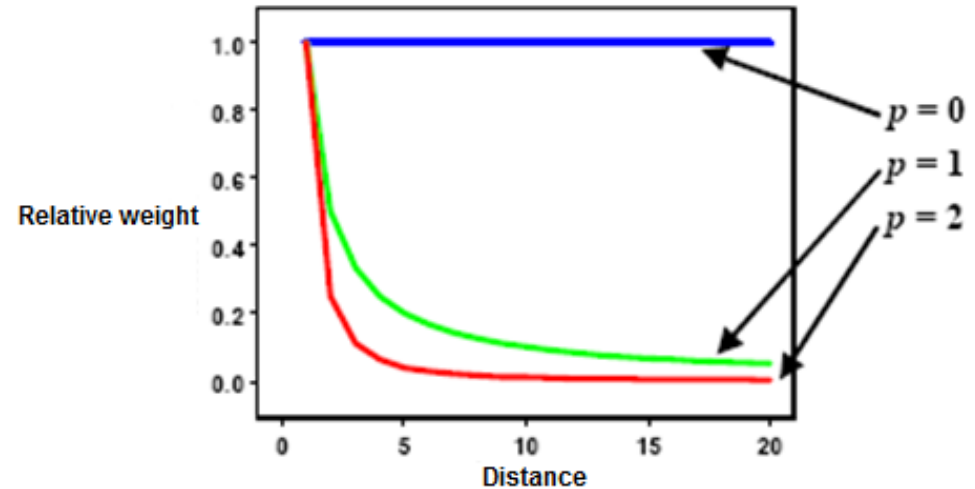
- optimal linear unbiased predictor using the variogram

Python packages: `scipy.interpolate`, `pykrige`

Inverse Distance Weighting (IDW)

Assumes spatial autocorrelation: nearby points are more similar

Weight of each known point decreases with distance from the target



The steepness of the distance-decay is controlled by the power p

→ $p = 0$ → equal weighting (unweighted average)

→ $p = 2$ → standard IDW (default)

→ large p → only nearest neighbour matters

p can be tuned via cross-validation to minimise prediction error

IDW Math

The interpolated value at target s_0 is a weighted average:

$$\hat{Z}(s_0) = \sum \frac{w_i \cdot Z(s_i)}{\sum w_i}$$

where the weight of point i is:

$$w_i = \frac{1}{d(s_0, s_i)^p}$$

$d(s_0, s_i)$ = Euclidean distance from target to sample point i

p = power parameter (typically 2)

IDW in Python

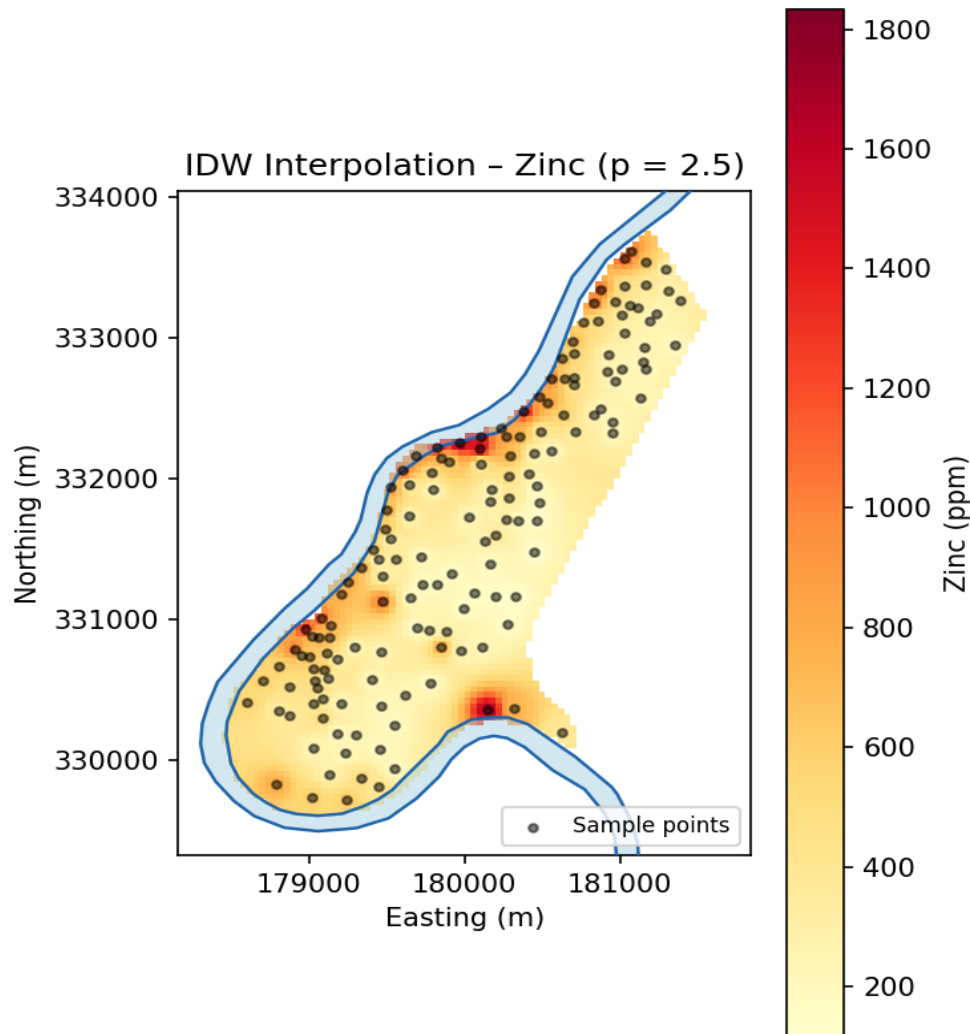
```
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist

df = pd.read_csv("meuse.csv")
x, y, zinc = df.x.values, df.y.values, df.zinc.values

# Build prediction grid
xi = np.linspace(x.min(), x.max(), 200)
yi = np.linspace(y.min(), y.max(), 200)
XI, YI = np.meshgrid(xi, yi)
grid_pts = np.column_stack([XI.ravel(), YI.ravel()])

# IDW (p = 2.5)
D = cdist(grid_pts, np.column_stack([x, y]))
D = np.where(D == 0, 1e-10, D) # avoid division by zero
W = 1.0 / D**2.5
W /= W.sum(axis=1, keepdims=True) # normalise weights
z_idw = (W @ zinc).reshape(XI.shape)
```

IDW Result – Raw Zinc



IDW of raw zinc ($p = 2.5$)

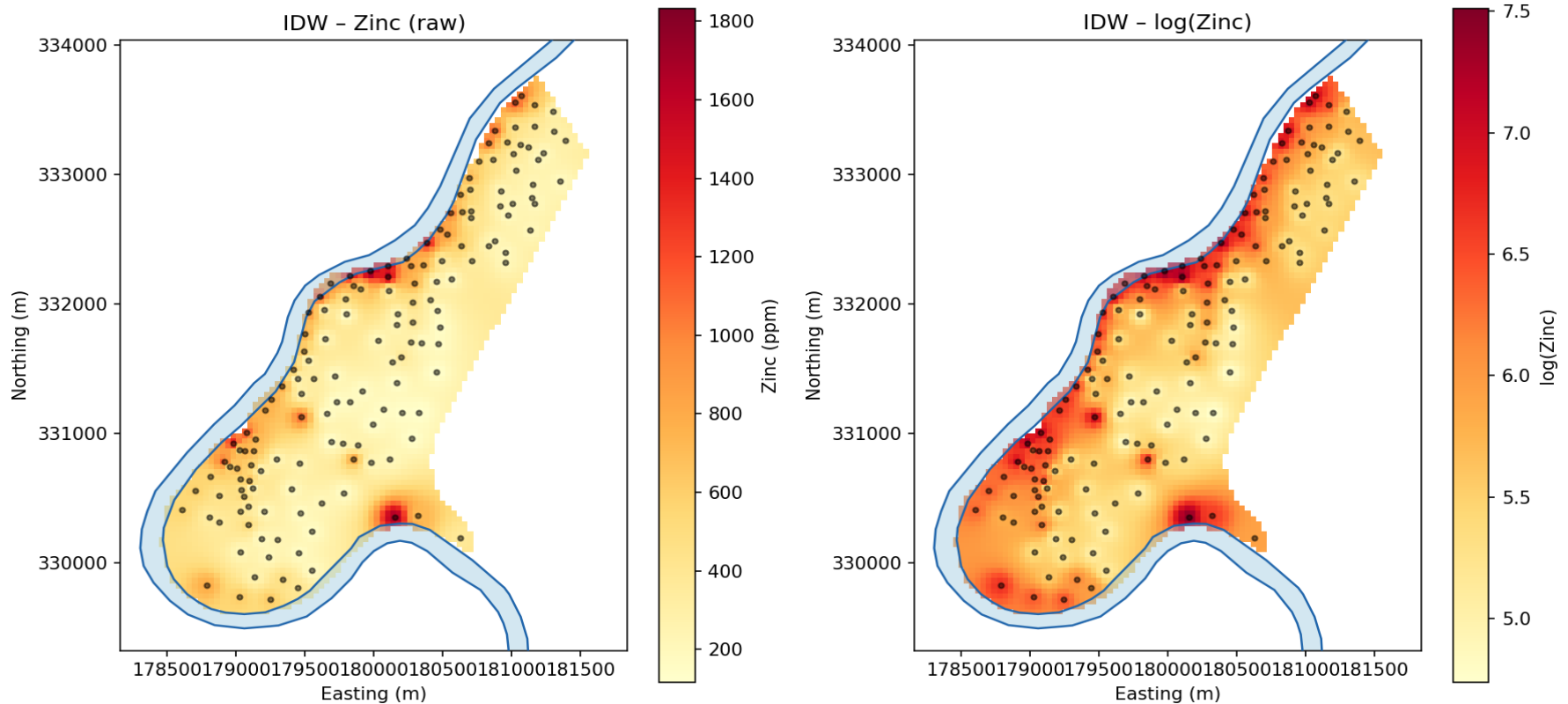
High zinc concentrations cluster near the river

Values decrease with distance from river

Some secondary hotspots visible — may reflect other factors

IDW assumes a smooth, continuous surface → may struggle with exponential gradients

IDW: Effect of the Log Transform



- Raw and log-transformed IDW look noticeably different
- IDW assumes smooth linear transitions — poor fit for exponential gradients
- Log transform can stabilize the variance before interpolation

Natural Neighbor Interpolation (NNI)

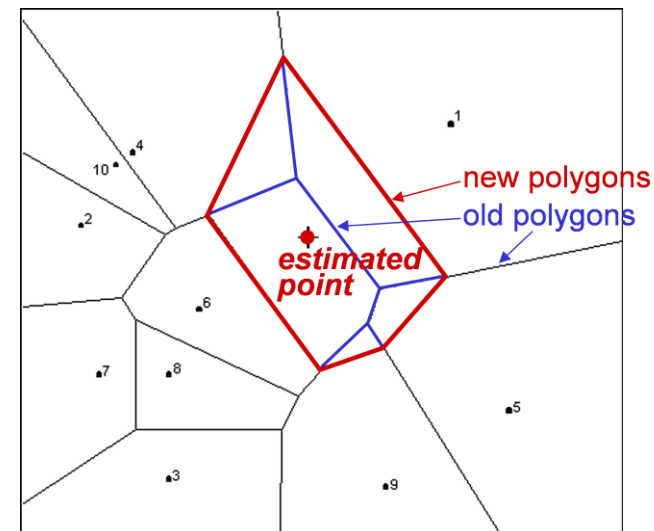
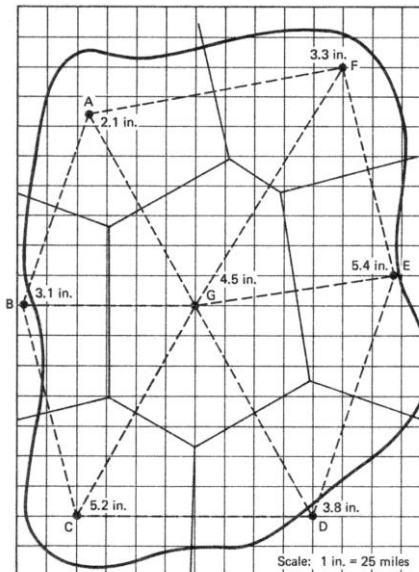
A variation on IDW where weights are based on area of polygon overlap

Algorithm:

1. Build Voronoi (Thiessen) polygons from sample points
2. Assume the whole polygon has the same value as its center
3. At target point s_0 , insert it into the Voronoi diagram
4. Weight each neighbor by how much area it "gives up" to s_0

Properties:

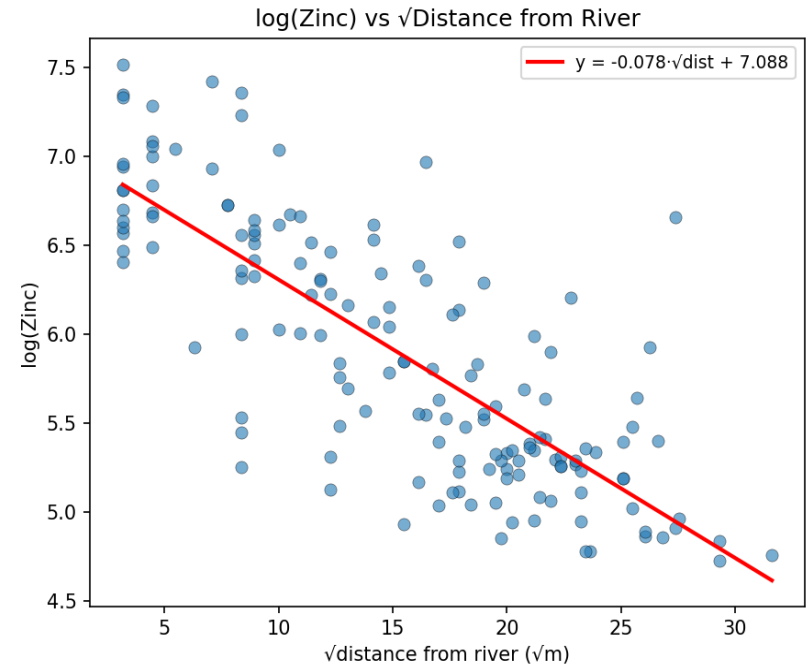
- Locally adaptive — honors the sampling density
- No user-defined power parameter



Interpreting Interpolation

Interpolation has no associated uncertainty or quality measure

→ Can resample or jiggle points to get a rough sense of stability



This biplot suggests a strong dependence of zinc on distance from the river

→ $\log(\text{zinc})$ decreases approximately linearly with $\sqrt{\text{distance}}$

Including a trend term (e.g., distance) can improve interpolation

Linear Model: $\log(\text{Zinc}) \sim \sqrt{\text{Distance}}$

```
import numpy as np, pandas as pd
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm

df = pd.read_csv("meuse.csv")
X = np.sqrt(df["dist.m"].values).reshape(-1,1)
y = np.log(df["zinc"].values)

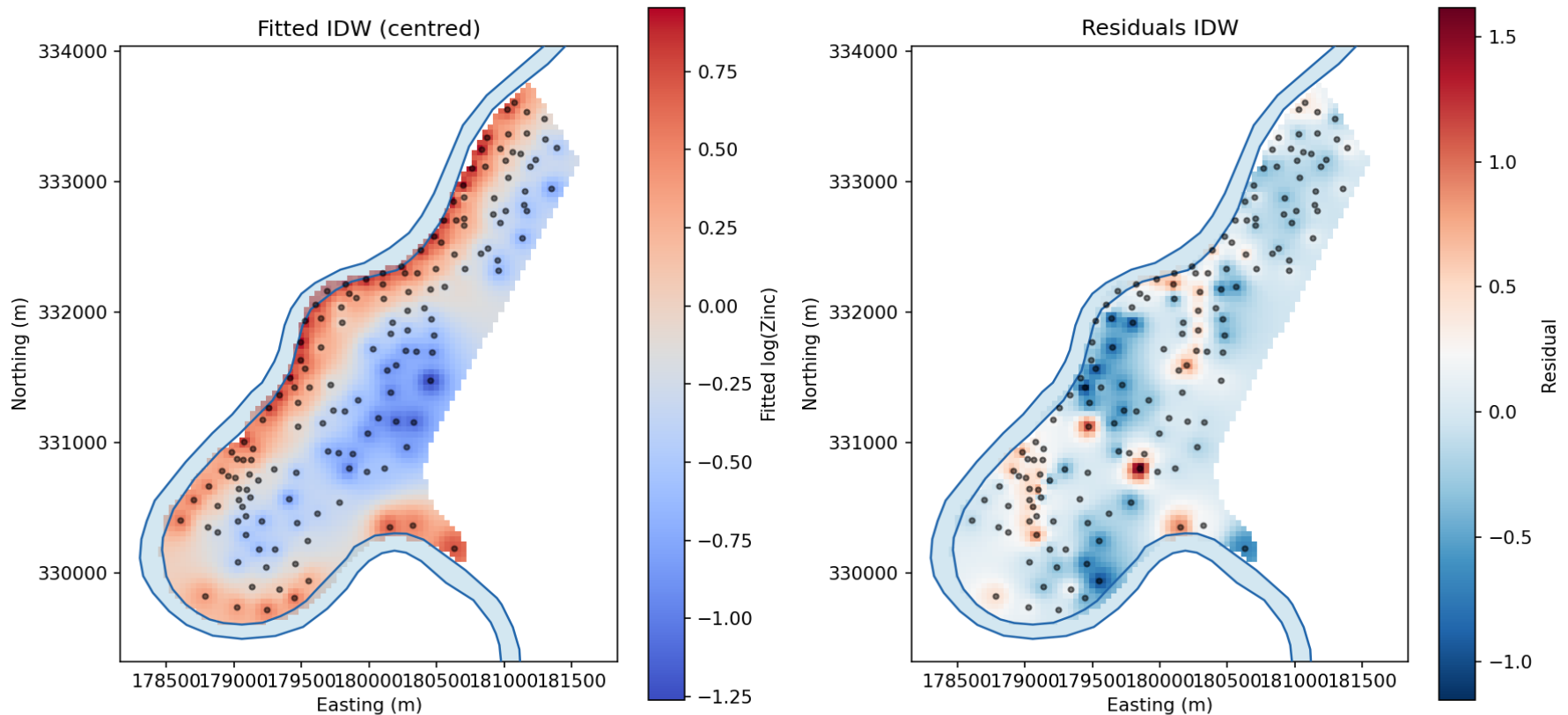
# Fit with statsmodels for ANOVA-style output
X_sm = sm.add_constant(X)
model = sm.OLS(y, X_sm).fit()
print(model.summary())

# Store fitted values and residuals
df["fitted"] = model.fittedvalues - model.fittedvalues.mean()
df["residuals"] = model.resid
```

$\sqrt{\text{dist.m}}$ is highly significant: $F = 270.6$, $p < 2.2e-16$

$R^2 \approx 0.64$ — distance explains ~64% of variance in $\log(\text{zinc})$

Fitted Values and Residuals (Spatial View)



Residuals show clear spatial structure → distance alone is not enough

Spatially correlated residuals → opportunity for kriging to improve the prediction

Accounting for Trends in Spatial Data

Basic IDW: only distance between points matters, not location itself

Parametric approach (like time series):

$$Z(s) = \textit{trend}(s) + \textit{spatial autocorrelation} + \textit{noise}$$

- Trend: a deterministic function of location (e.g., distance from river)
- Spatial autocorrelation: structure in the residuals — captured by a variogram
- Noise: unexplained variation

Variograms are the key tool for estimating spatial autocorrelation

Estimating Spatial Correlation: The Variogram

A variogram (= semivariogram) estimates spatial autocorrelation as a function of distance

Equivalent to:

→ correlogram / covariogram in other fields

In a single biplot, we cannot estimate correlation from one measurement per point

→ We need pairs of points at different separations

→ The variogram makes use of all pairs of points simultaneously

The variogram is central to geostatistics and kriging

Stationarity Assumption

Model: $Z(s) = m + e(s)$, where m is a constant mean

→ $E[Z(s)] = m$ (expected value is the same everywhere)

To estimate spatial correlation between Z values we need:

- a) Multiple measurements at each (x, y) point, OR
- b) Stationarity — so correlation depends only on the separation h , not the absolute location

With stationarity, all pairs of points separated by distance h contribute to estimating $\gamma(h)$

→ turns a small dataset into many distance-paired observations

Semivariance: Definition

The variogram $\gamma(h)$ is defined as:

$$\gamma(\mathbf{h}) = \frac{1}{2} E[(Z(\mathbf{s}) - Z(\mathbf{s} + \mathbf{h}))^2]$$

$\gamma(\mathbf{h})$ — semivariance at lag distance $\mathbf{h}$

$Z(\mathbf{s})$ — value at location $\mathbf{s}$

$Z(\mathbf{s}+\mathbf{h})$ — value at location $\mathbf{h}$ units away

In practice, we estimate $\gamma(\mathbf{h})$ from all pairs with separation $\approx \mathbf{h}$:

$$\widehat{\gamma(\mathbf{h})} = \frac{1}{2 \cdot N(\mathbf{h})} \sum (Z(\mathbf{s}_i) - Z(\mathbf{s}_j))^2$$

$N(\mathbf{h})$ = number of point pairs in distance bin $\mathbf{h}$

Building the Variogram

Step 1 — For every pair (i, j) of sample points:

- compute the separation distance $h = |s_i - s_j|$
- compute the squared difference $\frac{1}{2}(Z(s_i) - Z(s_j))^2$

Step 2 — Plot each pair as a point: $(h, \frac{1}{2}\Delta^2)$

- this is the variogram cloud

Step 3 — Average the cloud into distance bins

- this is the empirical (binned) variogram

Step 4 — Fit a parametric model to the empirical variogram

- Spherical, Gaussian, Exponential, Matérn, ...

Interpreting Semivariance

Completely random $Z(s)$: $\gamma(h) = \text{constant}$ for all h

→ Co-variation does not depend on distance (no spatial structure)

Spatially structured $Z(s)$:

→ $\gamma(h) \approx 0$ at small h — nearby points are very similar (autocorrelated)

→ $\gamma(h)$ increases with h until it levels off at the sill

Key variogram parameters:

- Nugget (c_0) — semivariance at $h \rightarrow 0$: measurement error + micro-scale variation
- Sill ($c_0 + c$) — semivariance plateau: total variance of Z
 - c denotes partial sill, i.e., amplitude of the spatial structure
- Range (a) — distance at which sill is reached: spatial correlation scale

"Error" in Variograms — Leading to Kriging

In OLS / GLM: correlated residuals are a problem

→ They violate independence assumptions

In spatial prediction: correlated residuals are an opportunity

→ The spatial structure of the residuals can be modelled via a variogram

→ The variogram is then used to improve predictions: kriging

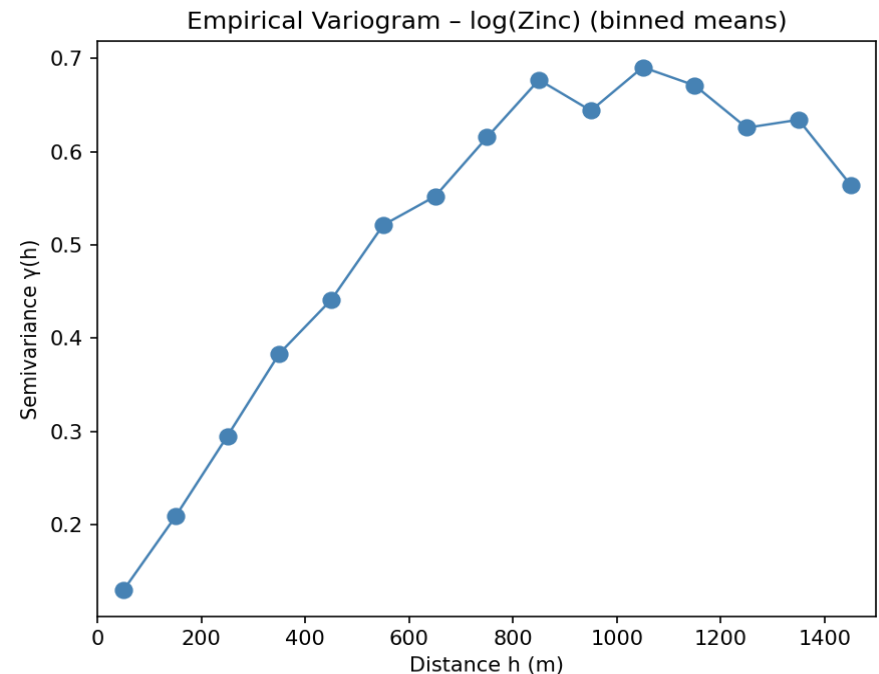
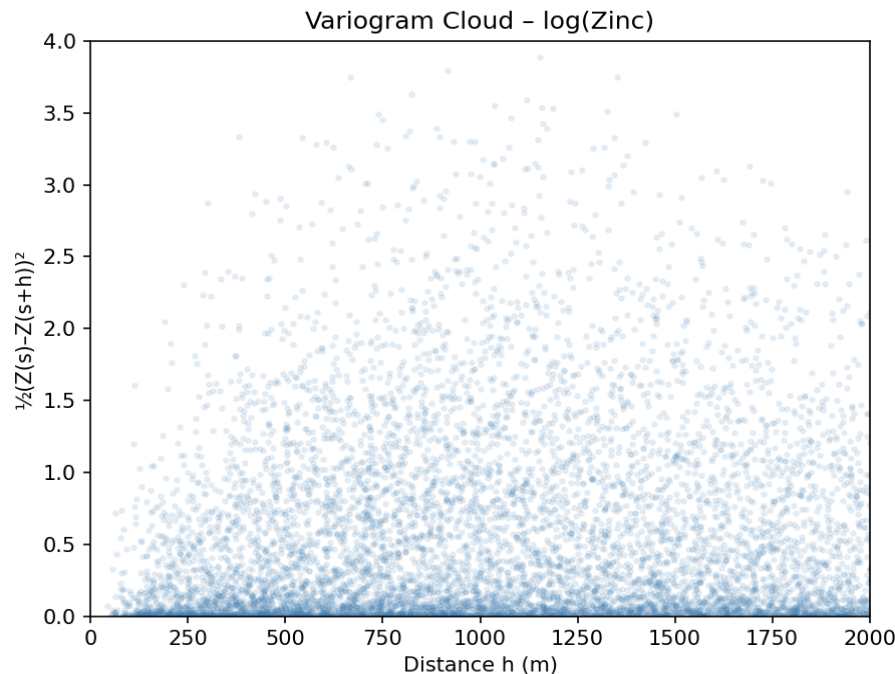
Kriging exploits spatial correlation in the residuals to make the best linear unbiased predictions (BLUP)

Unlike IDW, kriging also provides a measure of prediction uncertainty

Empirical Variogram in Python

```
import numpy as np
from scipy.spatial.distance import cdist

x, y, logz = df.x.values, df.y.values, np.log(df.zinc.values)
n = len(df)
h_list, g_list = [], []
for i in range(n):
    for j in range(i+1, n):
        h_ij = np.sqrt((x[i]-x[j])**2 + (y[i]-y[j])**2)
        g_ij = 0.5 * (logz[i] - logz[j])**2
        h_list.append(h_ij); g_list.append(g_ij)
# bin into distance classes to get empirical variogram
```



Variogram Modelling

Fit a parametric model to the observed empirical variogram

→ The empirical variogram is noisy; the model smooths and extrapolates it

Common variogram models:

Spherical — reaches sill at finite range (most common)

Exponential — asymptotically approaches sill

Gaussian — smooth near origin; strong short-range continuity

Matérn — flexible; generalises exponential and Gaussian

Power law — no finite sill; non-stationary

Each model embeds assumptions about how $\gamma(h)$ grows with h

Fitting a Variogram Model in Python

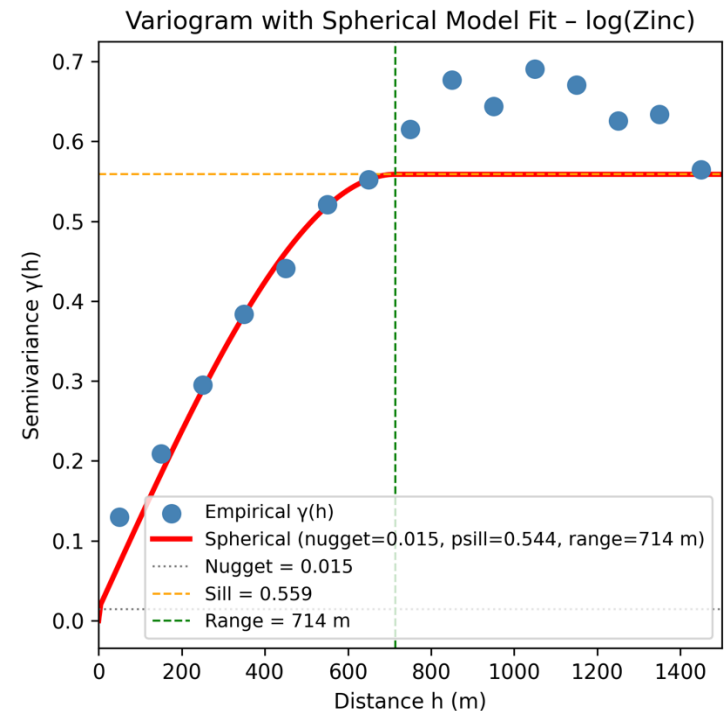
```
from pykrige.ok import OrdinaryKriging

# Fit variogram automatically
OK = OrdinaryKriging(
    x, y, log_z,
    variogram_model='spherical',
    nlags=40,
    weight=True,
    verbose=True, enable_plotting=True
)
```

Spherical model fit:

nugget = 0.015
psill = 0.544
range = 714 m

Range ≈ 714 m: zinc values are correlated up to ~ 714 m from each other



Kriging: Concept and Math

Kriging = Best Linear Unbiased Predictor (BLUP) of Z at unsampled s_0

General model: $Z(s) = \beta X + \varepsilon(s)$

X — predictor variables (e.g., distance from river)

β — regression coefficients

$\varepsilon(s)$ — spatially correlated residuals (captured by variogram)

The prediction at s_0 uses:

V — covariance matrix of Z (from variogram model)

v' — covariance vector between $Z(s)$ and $Z(s_0)$

Result: prediction + kriging variance (= uncertainty estimate)

Types of Kriging

Simple Kriging — known constant mean m

→ rarely used; mean almost never known

Ordinary Kriging — unknown but constant mean (most common)

→ mean estimated locally from nearby data

→ `pykrige: OrdinaryKriging`

Universal Kriging — mean varies as a polynomial of location

→ a.k.a. "trend surface" kriging

→ `pykrige: UniversalKriging`

Regression Kriging — mean is a linear model of covariates (e.g., distance from river)

→ most powerful when covariates explain most of the variation

Kriging in Python (pykrige)

```
import numpy as np, pandas as pd
from pykrige.ok import OrdinaryKriging

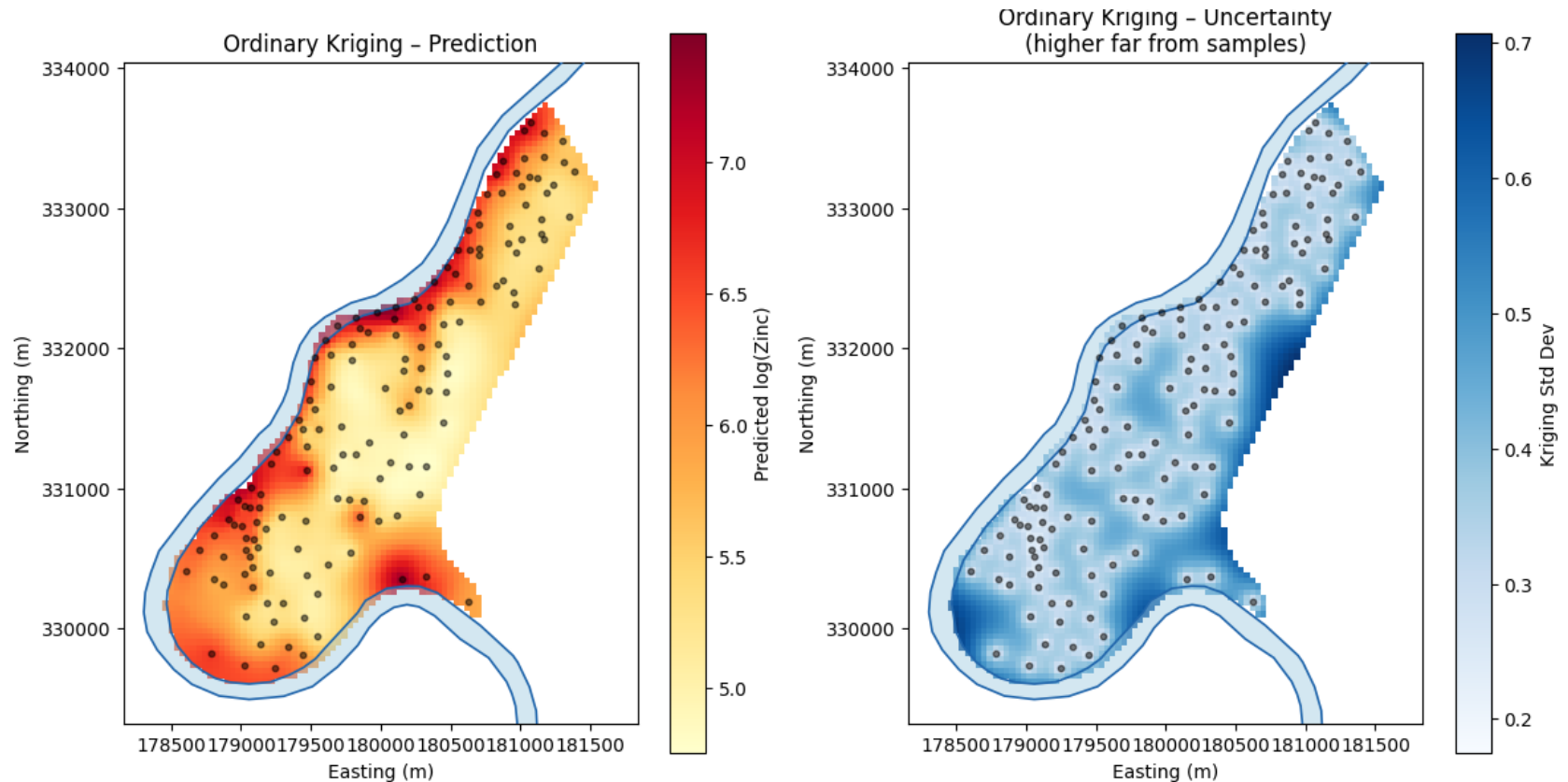
df = pd.read_csv("meuse.csv")
x, y = df.x.values, df.y.values
log_zinc = np.log(df.zinc.values)

# Prediction grid
xi = np.linspace(x.min(), x.max(), 200)
yi = np.linspace(y.min(), y.max(), 200)

# Fit variogram and predict
OK = OrdinaryKriging(
    x, y, log_z,
    variogram_model='spherical',
    variogram_parameters={'psill': psill, 'range': range_, 'nugget': nugget},
    verbose=True, enable_plotting=False
)

z_pred, z_var = OK.execute("grid", xi, yi)
# z_pred = predicted log(zinc) on grid
# z_var = kriging variance (= uncertainty2)
```


Kriging Results – Prediction and Uncertainty

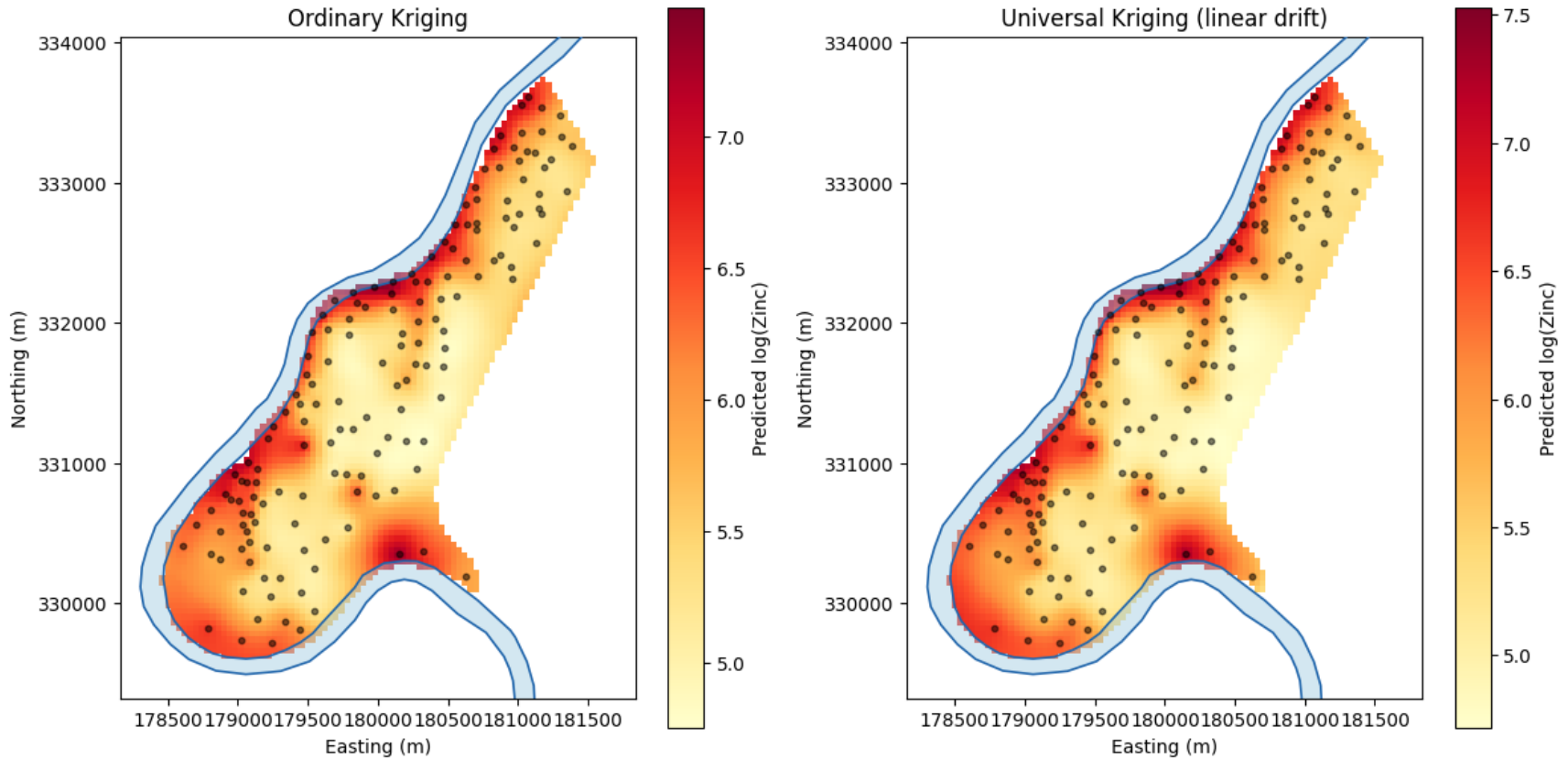


Left: predicted log(zinc) — smooth surface honoring the variogram structure

Right: kriging std dev — uncertainty increases far from sample points

Unlike IDW, kriging provides a rigorous estimate of prediction uncertainty

Trend Surface Analysis



- Trend surface = Universal Kriging with a polynomial of (x, y) as the drift
- Polynomial forms: linear (1st order), quadratic (2nd order), cubic (3rd order)
- *Left: Ordinary Kriging | Right: Universal Kriging with linear drift*

Today's learning outcome

- Apply IDW interpolation and explain the role of the power parameter p
- Describe natural neighbor interpolation and its geometry
- Compute and interpret an empirical variogram
- Identify nugget, sill, and range from a variogram plot
- Fit a parametric variogram model (Spherical, Gaussian, ...)
- Run Ordinary and Universal Kriging using pykrige